

相关框架调研

相关具身框架调研清单：

1. embodied-agents (mbodi): <https://github.com/mbodiai/embodied-agents>
2. embodied-agents (Automatika): <https://github.com/automatika-robotics/embodied-agents>
3. RoboOS: <https://github.com/FlagOpen/RoboOS>
4. HoloAgent: <https://github.com/HorizonRobotics/HoloAgent>
5. RoboNeuron: <https://github.com/guanweifan/RoboNeuron>
6. Dimos: <https://github.com/dimensionalOS/dimos>
7. rai: <https://github.com/RobotecAI/rai>

1. embodied-agents (mbodi AI)

仓库: <https://github.com/mbodiai/embodied-agents> | 文档: <https://api.mbodi.ai/docs>

简介：

把大型多模态/Transformer 模型以极少代码接入现有机器人技术栈的 Python 工具包，面向研究与原型开发。

架构：三大组件 **Agents / Data / Hardware**，每个再分为三种元模态 **Language(语言) / Motion(运动) / Sense(感知)**。所有 agent 暴露统一的 `act` 方法：

- Language Agent → 返回字符串
- Motor Agent → 返回 `Motion`
- Sensory Agent → 返回 `SensorReading`
- `AutoAgent` 可根据任务动态选择合适的 agent

调用支持本地/远程、同步/异步（基于 Gradio、httpx、LLM 客户端）；数据用 Pydantic 校验

- **技术栈**：Python、Pydantic、Gymnasium、HuggingFace、HDF5

- **机器人/仿真**: `SimRobot` (模拟)、`XArm` (实机)、`SimplerEnv`
- **成熟度**: ~290 stars; PyPI 最新版 **1.5.1 (2025-12-16)** ; 活跃但标注「实验性、可能有破坏性变更」

2. embodied-agents (Automatika Robotics + Inria)

仓库: <https://github.com/automatika-robotics/embodied-agents> | 文档: <https://automatika-robotics.github.io/embodied-agents/>

核心能力

一个 **ROS2 原生** 框架, 用于构建交互式具身物理智能体——不只是「聊天」, 而是能**理解、记忆、移动、操纵并适应环境**。核心概念: **组件图 (Component Graph)**: 以 AI 组件为节点, 通过 ROS2 Topic 连接成图。每个组件有 `inputs` / `outputs` (Topic) 与 `trigger` (触发条件), 由 `Launcher` 统一启动。**事件驱动 / 自指逻辑 (Self-Referential)**: 组件可依内/外部事件自我启停、重配置, 在云端与本地 ML 间热切换 (model client hot-swapping、fallback recovery)。底层依赖 **Sugarcoat (原 ROS Sugar)** 提供 ROS2 生命周期与事件管理。**语义记忆 / 向量库**: `memory` 组件 + **ChromaDB** 向量库 + `map_encoding` (层级化**时空记忆**) + `semantic_router` (语义路由)。**Cortex 规划器**: planner + events + ROS services。**内置组件**: `LLM` / `MLLM` / `VLM` / `vision` (检测) / `speechtotext` / `texttospeech` / `vla` / `map_encoding` / `semantic_router` / `memory` / `cortex` / `imagestovideo`。

- **技术栈**: Python、ROS2 **Humble+** (Humble/Iron/Jazzy)、MIT 许可
- **模型后端无关**: **Ollama**、**RoboML**, 及任何 **OpenAI 兼容 API** (vLLM、lmdeploy 等)
- **机器人/硬件**: 含 **LeRobot** 集成 (gRPC transport), 针对边缘设备与 **NVIDIA Jetson** 优化

3. RoboOS (BAAI / FlagOpen)

仓库: <https://github.com/FlagOpen/RoboOS> | 论文: arXiv 2505.03673

核心能力

RoboOS 是**首个开源的具身操作系统**, 基于「**大脑-小脑**」(**Brain-Cerebellum**) **分层架构**, 目标是把机器人智能从单体 (single-agent) 推向**集群智能 (swarm intelligence)**, 实现跨本体 (cross-embodiment) 与多机协作。它针对三大痛点: 跨本体适配差、任务调度低效、动态纠错弱。端到端 VLA (OpenVLA/RDT/Pi-0) 长程规划弱; 分层 VLA (Helix/Gemini-Robotics/GR00T-N1) 缺乏跨本体与多机协调——RoboOS 填补此空白。

核心组件: **Embodied Cloud Model (云端大脑)**: 多模态大模型 (即 RoboBrain), 负责全局感知与高层决策/规划; **Cerebellum Skill Library (小脑技能库)**: 模块化、即插即用的技能工具包, 执行具体动作; **Real-Time Shared Memory (实时共享记忆)**: 时空同步机制, 协调多机状态。

- **跨异构本体**: 单臂、双臂、人形、轮式 (demo 用 Realman 单臂 + Agilex 双臂)

- **配套大脑模型**：RoboBrain 2.0 / 1.0
- **成熟度**：~590 stars, 88 forks；活跃维护中

4. HoloAgent (地平线 Horizon Robotics)

仓库：<https://github.com/HorizonRobotics/HoloAgent> | 项目页：
https://horizonrobotics.github.io/robot_lab/holoagent/

核心能力

HoloAgent 是地平线推出的**通用机器人统一具身智能体框架**，把 LLM Agent 的「推理→调用工具→观察反馈→修正」闭环从数字世界扩展到真实物理机器人。核心问题：现有具身系统的操作、导航、空间理解等能力多为**孤立模块、松耦合**，难以统一调度执行长程任务。**核心组件**：**Embodied AgentOS** — 把自然语言指令转成可执行的「技能图 (skill graph)」，调度机器人资源、监控执行、依运行时反馈触发澄清或重规划（闭环执行）；**3D Spatial Memory** — 用三维空间表征为机器人推理提供物理世界 grounding，支撑长程任务与在线记忆更新；**Embodied Skills** — 把 Agent 决策连接到可执行的导航/操作技能；**FSR-VLN** — 导航组件：分层多模态场景图 + 快慢推理的视觉语言导航

特点

- **技术栈**：C++ / Python / CMake；ROS 2 Humble + Nav2 + FastLIVO2 里程计/重定位；感知用 OpenCLIP + SAM；VLM 用 Azure OpenAI (gpt-4/4o)
- **支持机器人/仿真**：实机主用 **Unitree-G1 人形机器人**；仿真支持 **Habitat-Sim** (HM3DSem)。demo 含跨机器人导航与舞蹈协同、长程移动操作、主动探索、人形指令交互、陪伴/导览
- **成熟度**：~201 stars；创建于 2025-11，最后推送 2026-06（活跃）
- **发布状态**：FSR-VLN 代码已开源；**HoloAgent-0 完整代码尚未发布** ("under preparation")，目前仓库仅含 `fsr_vln/` 和 `nav_agent/`

5. RoboNeuron

仓库：<https://github.com/guanweifan/RoboNeuron>

核心能力

RoboNeuron 是一个**面向具身智能的 ROS2 原生中间件层 (middleware layer)**，连接基础模型 (VLA / LLM agent) 与机器人执行系统。核心问题：**agent 工具 API 与机器人中间件 (ROS2) 之间的不匹配**，避免为每个机器人写难以复用的定制 wrapper。核心组件：架构上它把 **agent 侧编排 (orchestration)** 与 **ROS2 执行** 解耦，支持两种模式：1. 低延迟的「工具 → ROS 直控」2. 闭环「感知 → VLA 推理 → 控制」管线。通过 **MCP (Model Context Protocol)** 暴露能力，并能直接从 ROS 消息 schema **自动生成** agent 可调用的工具。

特点

- **技术栈**：Python 3.12+、ROS 2 Jazzy（兼容 Humble）、UV 包管理、MCP（客户端用 CLine / OpenClaw）
- **支持 VLA 模型**：`dummy`、`openvla`、`openvla-oft`（共享 adapter 接口，独立 Python 3.10 运行时隔离 PyTorch/flash-attn 依赖）
- **机器人/仿真**：Franka Panda、Franka Research 3（含 URDF）；演示用 Isaac Sim 及真实世界 RGB 相机抓取

6. Dimos (Dimensional Inc.)

仓库：<https://github.com/dimensionalOS/dimos> | **官网**：<https://dimensionalos.com/> |

License：Apache-2.0

核心能力

DimOS 自称「面向物理空间的智能体操作系统 (agentic operating system for physical space)」——一个生成式机器人通用 SDK/OS。核心特征是**不依赖 ROS**，纯 Python 即可为人形、四足、无人机等异构硬件构建可跨本体复用的「物理应用」，并用自然语言 (vibecode) 驱动机器人，把 AI agent 直接接入硬件数据流（感知、空间记忆、控制环、电机驱动）。

核心组件：Modules：机器人上的子系统，通过类型化 `In[]` / `Out[]` 流交换标准消息；

Blueprints：声明式装配/连线机器人栈，用类型安全的 `autoconnect()` 按 `(name, type)` 自动连接流；**Transports**（可插拔）：LCM、SHM、DDS、ROS 2；**Agents**：作为原生 module 运行，订阅任意嵌入式流，底层用 **MCP**；数据流基于 **RxPY** 响应式流，支持背压与错误处理；**Skills**：让 agent 调用硬件物理函数，如 `grab()`、`follow_object()`。理念为「神经符号 (neurosymbolic) 编排」——LLM 做空间推理/规划，机器人状态/动作原语作为函数

特点

- **功能**：导航与建图（SLAM、避障、A* 规划、探索）、感知（检测器、3D 投影、VLM、音频）、MCP 智能体控制、空间记忆（时空 RAG、物体恒存性）
- **技术栈**：Python (~92%)、Rust、C++、Nix/NixOS flakes、Docker、CUDA；

7. rai (RobotecAI)

仓库：<https://github.com/RobotecAI/rai> | **文档**：<https://robotecai.github.io/rai/> | **License**：Apache-2.0 |

核心能力

RAI 是面向「物理AI / 具身智能」的 **vendor-agnostic（厂商无关）多智能体框架**。它把生成式 AI（LLM/VLM）接入机器人技术栈，让机器人具备自然语言人机交互、复杂动作执行、场景化任务、日志总结与语音交互能力。核心是**深度集成 ROS 2**：将 LLM agent 与现有 ROS 2 工具/话题/动作打通，无需重写底层栈即可叠加「智能层」。

核心组件： `rai_core` — 多智能体系统、人机交互、多模态核心； `rai_whoami` — 从文档/图片/URDF 提取机器人「身体认知」(embodiment)； `rai_asr` / `rai_tts` — 语音转文字 / 文字转语音； `rai_sim` — 连接仿真环境； `rai_bench` — agent/模型/工具/仿真器基准测试套件； `rai_perception` — 开放集目标检测（Grounded SAM 2）； `rai_nomad` — NoMaD 导航集成； `rai_finetune` — 在具身数据上微调 LLM

特点

- **技术栈：** Python； ROS 2 **Jazzy & Humble**； Ubuntu 24.04/22.04； Python 3.12/3.10
- **仿真器：** O3DE（官方主推）、 `rai_sim` 抽象层； 论文测试了机械臂与拖拉机仿真
- **支持机器人（demo）：** Husarion ROSbot XL（含数字孪生）、 Franka Panda 机械臂、 RB-KAIROS 移动操作臂、 果园自主拖拉机

调研总结

框架	核心定位	主要特点	对我们有用的点 /
mbodi embodied-agents	多模态模型接入机器人栈的 toolkit	将系统分成 Language / Motion / Sense agents；支持本地或远程 act / async_act；Language 后端支持 OpenAI、Anthropic、Gemini、Ollama、vLLM 等，Motor Agent 支持 OpenVLA，Sense Agent 支持 Depth Anything、YOLO、SAM2 等；也支持自动数据记录。	优点是模型接入简单、后端丰富；model wrapper / agent toolkit，Pipeline、Verifier、Recovery、环。
Automatika embodied-agents / EMOS	ROS2-based 交互式物理智能体框架	定位为 ROS2 框架，用于构建能够理解、记忆并基于环境上下文行动的 interactive physical agents。ROS index 里也将其描述为 EMOS 的 intelligence layer，用于让机器人理解、移动、操作并适应环境。	优点是 ROS2 工程化和真实机器、品/机器人智能层，缺少面向 rese Pipeline、Benchmark、AgentT 出。
RoboOS	跨 embodiment、多机器人协作的具身操作系统	提出 Brain-Cerebellum 层级架构，目标是解决端到端 VLA 长程规划弱、层级 VLA 缺跨 embodiment 和多智能体协作的问题；定位是 cross-embodiment and multi-agent collaboration。	优点是多机器人、跨身体协作和、偏多机器人 OS / swarm intellige Agent 长程任务训练、评测、Ver 闭环的通用框架。
HoloAgent	面向真实机器人的统一 AgentOS + 3D Spatial Memory	HoloAgent-0 将系统组织为 Embodied AgentOS、3D spatial memory 和 embodied skills；AgentOS 将语言指令转成 executable skill graphs，调度机器人资源，监控执行，并根据 runtime feedback 触发 clarification 或 re-planning。	优点是空间记忆、skill graph、真整；不足是更偏 deployment/ru benchmark、多后端评测、Ager 训练接口。
RoboNeuron	MCP + ROS/ VLA 的机器人部署中间层	目标是把 LLM/VLA 的认知能力与 ROS 的实时执行骨架连接起来；用 MCP 作为 semantic bridge，让 LLM 动态编排 ROS 工具；通过 ROS 通信接口解耦 sensing、reasoning、control，并自动将 ROS messages 转成 callable MCP functions。	很适合作为我们的 ROS2Runtime Layer 参考；不足是主要解决 LLI 接，缺少高层 Pipeline、Memor Recovery、Evaluation 和 Data f
Dimos	面向 physical space 的 agentic operating system	定位为 generalist robotics 的现代操作系统；强调 Python-first、no ROS required，可以在 humanoid、quadruped、drone 等硬件上构建 physical applications；GitHub 描述也强调通过自然语言构建多智能体系统，并连接 camera、lidar、actuator 等物理输入输出。	优点是 agent-native、硬件抽象；不足是更像 physical OS / SDK，务评测、过程验证、失败恢复和心。
RAI	ROS2 机器人上的 vendor-agnostic multi-agent framework	用于 Physical AI robotics，支持 ROS2 tools、复杂动作、场景执行、日志总结、语音交互等。论文中将 RAI 定位为创建 embodied Multi-Agent Systems 的框架，集成 LLM、robotic stacks、external APIs、HRI，并提供 ROS2、仿真和实体机器人部署。	优点是多 Agent、ROS2、HRI、1 完整；不足是 Agent 内部环境模：定义，论文也提到多 Agent 对 er mission state 理解可能不一致，success detection 也存在不稳定

总结：

1. 现有框架主要解决 LLM/VLM/VLA 与机器人系统之间的连接问题，例如通过 ROS2、MCP、tool calling、agent connector 或 hardware SDK，把大模型能力接到机器人 perception、navigation、manipulation 和 control 模块上；
2. 多数框架偏工程部署或机器人操作系统，例如 RoboNeuron 侧重 MCP-ROS/VLA 桥接，Dimos 侧重 physical OS，RAI 和 Automatika 更偏 ROS2 机器人智能层，HoloAgent 更偏真实机器人 AgentOS，RoboOS 更偏跨 embodiment 和多机器人协作；
3. 现有框架基本没有记忆或状态管理；
4. 现有框架对过程验证和失败恢复支持不足，基本没有或者支持很少；
5. 现有框架基本是**同步、开环**的技能调用，缺少闭环loop和执行验证；
6. 现有框架难以下游应用，基本是面向demo的，而不是基础的、可复用的框架、并且难以评测。

我们的创新：

1. **Pipeline和推理流程编排**，6 个框架里只有 automatika 用「组件图」接近，但它是**事件驱动**的交互式数据流，不是「观察—规划—执行—验证—更新」的**长程任务推理循环**。
2. **Verifier验证，同步异步支持**
3. **完整的记忆支持**
4. **流程Pipeline/Agent决策/运行三者解耦**，方便复用，可以作为基础框架类似transformers
5. **Benchmark/评测支持**，可以真正评估通才智能体能力。Embodiedbench, robocasa, behavior-1k（拆分，浙大）
6. 真机支持（课题三）

(1,2,3)我们先立框架，确认接口，然后分配任务

基于现有框架来实现我们的框架：

你的模块	推荐来源 / 借鉴对象	怎么落地
Agent Core （观察-规划-执行-验证-更新）	mbodi Agent.act() 三态 + automatika cortex 规划器	以 mbodi 的统一 agent 接口为基类，observe/plan/act/verify/update 五
Pipeline （可编排推理流程）	automatika 组件图 + trigger 事件 / 或自研 LangGraph 式 DAG	把「观察→规划→执行→验证→更新」可声明的有向图节点，节点间传 AgentContext
LLM Backend Registry	rai config.toml + mbodi model_src 多后端	一个 register(name, client) 工厂，运按配置选 OpenAI/Ollama/vLLM； automatika 的 热切换/fallback 值得
Skill Backend Registry	RoboNeuron （schema→工具自动生成 + runtime 隔离） + mbodi OpenVlaAgent	抽象 SkillBackend.invoke(skill_call, action_chunk，分别实现 VLA / 规则器 / 规划器 / 工具 API 后端
Memory System	automatika ChromaDB 语义时空记忆 + map_encoding / HoloAgent 3D 记忆 / mbodi HDF5 持久化	短期：episode buffer；长期：向量库索引增强；mbodi Recorder 负责落盘与
Environment Interface	rai rai_sim 抽象 / mbodi SimRobot +SimplerEnv / Dimos Transport	统一 step(action) -> (obs, state, reward, done, success)，gym 式；适配器分：真 benchmark 与真机
Runtime （同步/异步/多环境调度）	Dimos Blueprint + RxPY 响应式 / rai launch	Runtime 负责 episode/session 生命周期、日志、并行 env、评测；与 Pipeline 一接口解耦
Verifier （子任务/整体完成判定）	自研（各框架都弱）；可借 LLM-as-judge + 环境 success signal	双路验证：环境 success 硬信号 + LLM VLM 软判定，失败触发重规划/重试

我们的框架：

Overview整体流程

本项目拟构建一个面向长程具身任务的可扩展 Agent 框架。如[整体框架图](#)所示，整体系统由 Pipeline、Agent Core、LLM Backend Registry、Memory System、Skill Backend Registry、Environment Interface 与 Runtime 组成，形成从任务理解、上下文构建、子任务规划、技能调用、动作执行、状态反馈到结果验证的闭环。

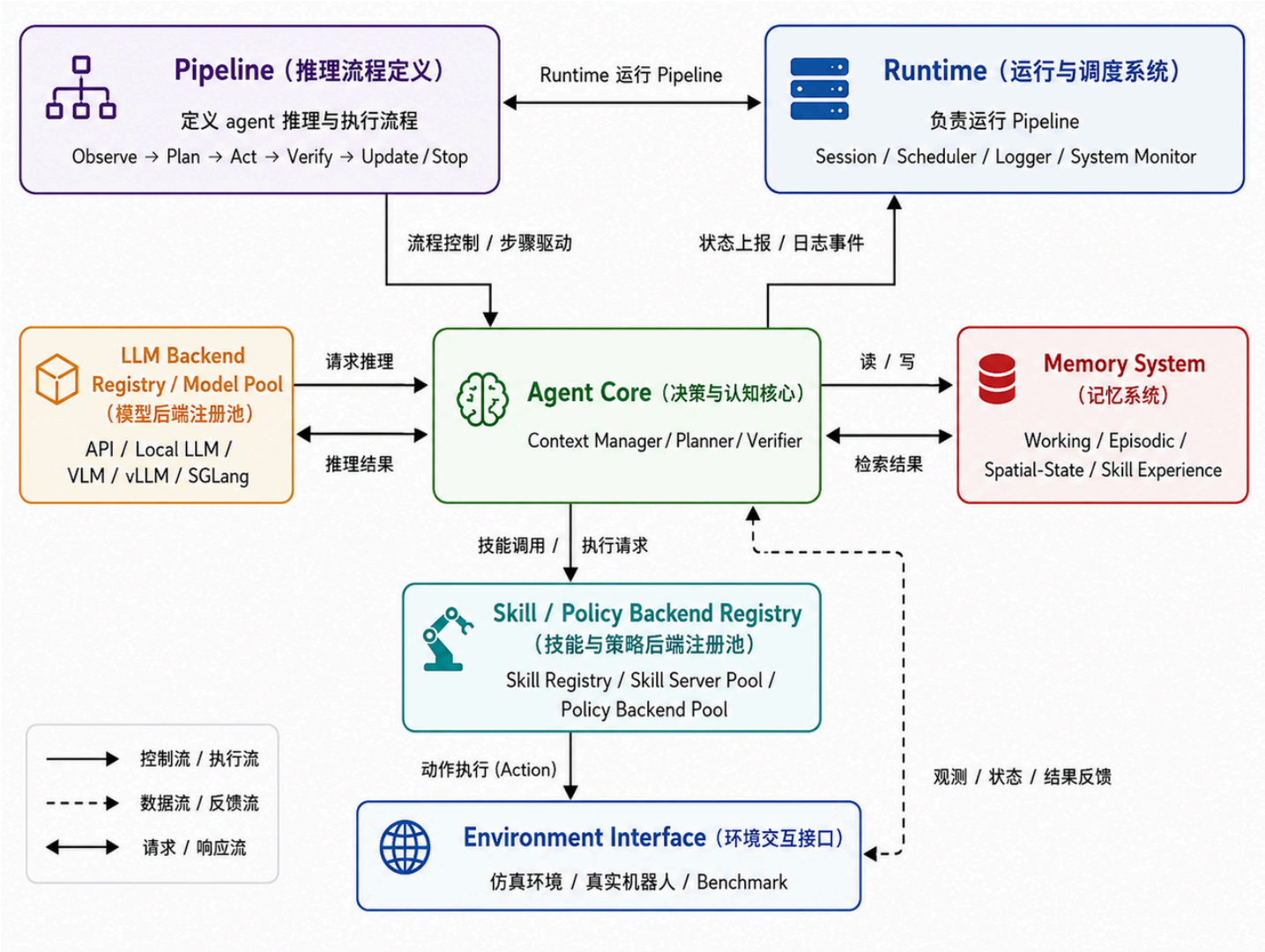
整体执行流程如下：

- a. 用户或 benchmark 输入任务目标、环境配置与评测参数；
- b. Runtime 启动并调度 Pipeline，创建 episode/session；
- c. Pipeline 按照预定义推理流程驱动Agent Core执行观察、规划、执行、验证与更新；
- d. Agent Core 通过Context Manager 汇总当前观测、任务状态、历史记忆和可用技能，并调用 Planner 生成下一步子任务或技能调用；

- e. Agent Core 通过 LLM Backend Registry 调用不同 LLM/VLM 后端完成任务推理、状态判断或结果验证；
- f. Agent Core 将高层决策转化为标准化 skill call，并提交给 Skill Backend Registry；
- g. Skill Backend Registry 根据 skill 类型和后端配置，路由到对应的 Skill Server 或 VLA/Policy Backend，生成动作或 action chunk；
- h. Environment Interface 执行动作，并返回 observation、state、reward、done、success signal 等反馈；
- i. Pipeline 将执行结果写入 Memory System，并由 Verifier 判断当前子任务或整体任务是否完成；
- j. 若验证成功，则进入下一子任务或终止；若失败，则触发重新规划或重试，形成持续闭环。

框架特点

- **整体框架模块化、可插拔、可扩展：** Pipeline 可自定义推理流程，Agent Core 可自定义调用模型等，Runtime 可切换同步/异步/多环境运行方式，LLM 与 Skill 后端可通过 Registry 动态切换，环境侧可适配仿真平台 benchmark 或真实机器人
- **流程、决策与运行解耦：** Pipeline 只负责定义“观察—规划—执行—验证—更新”的推理逻辑，Runtime 负责具体执行、调度、日志和评测。二者通过统一接口组合，使同一 Pipeline 可以运行在不同 Runtime 上，也使不同任务流程可以复用同一运行系统。同时高层任务推理与底层动作生成相互独立，便于接入不同类型的具身模型。
- **标准化接口设计：** 通过 AgentContext 等统一数据结构连接各模块，减少模块耦合，提高系统可维护性和可复用性
- **兼容学习型与非学习型技能：** Skill / Policy Backend Registry 既可以接入学习型 VLA / policy model，也可以接入规则控制器、传统规划器、检索模块或外部工具 API，使系统能够融合神经模型、符号工具和工程控制模块。
- **记忆增强的闭环决策：** 可结合记忆增强，功能丰富



整体框架逻辑关系图